Linked List Operation Algorithms

Singly Linked List:

Traversing the linked list

(Traversing a Linked List) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of LIST. The variable PTR points to the node currently being processed.

1. **Set PTR := START.** [Initializes pointer PTR.]
2. **Repeat Steps 3 and 4 while PTR ≠ NULL.**
3. **Apply PROCESS to INFO[PTR].**
4. **Set PTR := LINK[PTR].** [PTR now points to the next node.] [End of Step 2 loop.]
5. **Exit.**

Insertion in Linked List

Insertion at Beginning

INSFIRST(NEWNODE, START, ITEM)

This algorithm inserts ITEM as the first node in the list.

1. **[OVERFLOW?]** If **NEWNODE = NULL**, then: Write: **OVERFLOW**, and **Exit**.
2. **Set NEWNODE -> INFO := ITEM.** [Copies new data into new node]
3. **Set NEWNODE -> LINK := START.** [New node now points to original first node.]
4. **Set START := NEWNODE.** [Changes START so it points to the new node.]
5. **Exit.**

Insertion at Location

INSLOC(NEWNODE, START, ITEM, LOC)

1. **[OVERFLOW?]** If **NEWNODE = NULL**, then: Write: **OVERFLOW**, and **Exit**.
2. **[Check if Inserting Node at Begin?]**

   If **LOC = 1**, then:

   Call procedure **INSFIRST(NEWNODE, START, ITEM)**

   and **EXIT**
3. **[Find Location for Insertion]**

   Temp:=FINDLOC(LOC, START)
4. **SET NEWNODE -> INFO :**= **ITEM.** [Copies new data into new node]
5. **SET NEWNODE -> LINK := Temp -> LINK** [New node now points to node at next to the location.]
6. **SET Temp -> LINK := NEWNODE** [Node at LOC – 1 position points to NEW node.]
7. **EXIT**

PROCEDURE:

FINDLOC(LOC, START)

1. **SET PTR** := **START**
2. **[LOOP] K** := 1 to **K <= LOC – 1**

   PTR := PTR -> LINK

   K := K + 1
3. **Return PTR**
4. **EXIT**

Insertion at END

INSEND(NEWNODE, START, ITEM, LAST)

1. **[OVERFLOW?]** If **NEWNODE = NULL**, then: Write: **OVERFLOW**, and **Exit**.
2. **[CHECK Linked list is empty or Not]**

   If **START** = **NULL,** then:

   Call procedure **INSFIRST(NEWNODE, START, ITEM)**

   and **EXIT**
3. **Set NEWNODE -> INFO := ITEM.** [Copies new data into new node]
4. **Set NEWNODE -> LINK := NULL.** [New node now points to NULL.]
5. **Set LAST ->LINK := NEWNODE.** [Changes last node points to the new node.]
6. **Set LAST := NEWNODE.** [Changes LAST so it points to the new node.]
7. **Exit.**